DESENVOLVIMENTO ACELERADO COM IA

Engenharia de Contexto com IA

Do system prompt ao orquestrador de agents — como dar à IA o contexto que ela precisa para acertar.

Adriano Santos · Treinamentos

System prompt: o contexto que chega antes do seu prompt

Toda chamada para um LLM tem três camadas. O system prompt é a primeira — e é onde mora o poder.

System Prompt

Quem é o assistente

Quais regras seguir

Que formato usar

O que NÃO fazer

→ Você controla isso

User Prompt

O que o usuário digitou

"Faça um CRUD de tarefas"

"Corrija esse bug"

"Explique essa função"

→ O usuário digita

Output

A resposta do modelo

Influenciada por system + user

Sem system: genérica

Com system: especializada

→ O modelo gera

Quem escreve o system prompt controla a qualidade da resposta. No bolt.diy, alguém já escreveu. No seu projeto, você precisa escrever.

A IA não erra porque é burra. Ela erra porque está cega.

O modelo não sabe nada sobre o seu projeto. Ele sabe programar, mas não sabe o contexto do que você está construindo.

Arquitetura do projeto

não sabe se usa microserviços ou monolito

Decisões técnicas passadas

não sabe por que escolheram PostgreSQL e não MongoDB

Bugs já resolvidos

vai sugerir soluções que já falharam antes

Padrões do time

não sabe que vocês usam repository pattern

Contexto de negócio

não sabe o que "PIX" significa no seu sistema

Código existente

não conhece as funções que já existem no projeto

A IA precisa de óculos. O Harness é o par de óculos que dá contexto ao modelo sobre o seu projeto.

Por que a manutenção com IA parece um jogo de whack-a-mole

Sem contexto, cada correção é um tiro no escuro. O modelo corrige aqui, quebra ali.
Você apaga um incêndio e outro aparece.

1

Pede pra corrigir um bug

2

IA corrige o bug

3

Novo bug aparece em outro lugar

4

Pede pra corrigir o novo bug

5

IA corrige, mas quebra algo mais

6

Volta ao passo 1

Esse ciclo só para quando o modelo enxerga o contexto completo do projeto. Não quando ele acerta mais, mas quando ele para de errar por causa de coisas que já deveria saber.

Harness: o system prompt que evolui com o projeto

O system prompt do bolt.diy é fixo. Ele funciona para gerar apps do zero. Mas uma software house precisa de algo que mude a cada sprint, que aprenda com cada decisão, que capture o conhecimento do time.

System Prompt (fixo)

Escrito uma vez

Não muda com o projeto

Genérico por design

Bom para: **geração do zero**

Harness (evolutivo)

Evolui a cada sprint

Captura decisões do time

Específico do seu projeto

Bom para: **geração + manutenção**

O Harness é um system prompt que cresce junto com o projeto. Arquivos como CLAUDE.md, specs, plan.md e rules formam esse contexto vivo.

Os artefatos que dão memória e contexto ao agent

Cada um desses arquivos é uma peça do Harness. Juntos, formam o contexto que transforma um LLM genérico em um especialista no seu projeto.

CLAUDE.md
Identidade e regras do projeto

specs/
Documentação de features e APIs

plan.md
Plano de execução com tarefas

rules/
Regras por domínio e contexto

.cursorrules
Regras do editor/IDE

agents/
Agentes especializados

skills/
Skills injetáveis no contexto

hooks/
Automações em eventos git

context/
Arquivos de contexto seletivo

Hoje você vai criar esses artefatos. Cada um que você escreve é contexto permanente que o agent nunca mais vai precisar perguntar.

Hoje você constrói o contexto que faltava

O que muda quando o agent recebe o contexto certo?

Sem contexto

"Não sei como está estruturado"

"Vou assumir um padrão genérico"

"Não conheço os padrões do time"

"Vou reescrever do zero"

"Quebrou? Não sei por quê"

Com Harness

"Li a spec e o CLAUDE.md"

"Vou seguir o padrão do projeto"

"Já sei quais regras aplicar"

"Vou evoluir o código existente"

"Testes passam de primeira"

O modelo é o mesmo dos dois lados. A diferença é o contexto. E contexto se constrói. É exatamente isso que vamos fazer hoje.

A IA não pensa: ela prediz

Uma LLM não entende o que você pediu. Ela calcula qual é a sequência de palavras mais provável para completar o que você começou, baseada em tudo que foi treinado. Quando você pede "crie um app de gerenciamento de projetos", ela retorna o resultado mais estatisticamente provável para essa entrada: o centro da distribuição de todos os apps que existem na internet.

Entrada (prompt)

Modelo calcula probabilidades

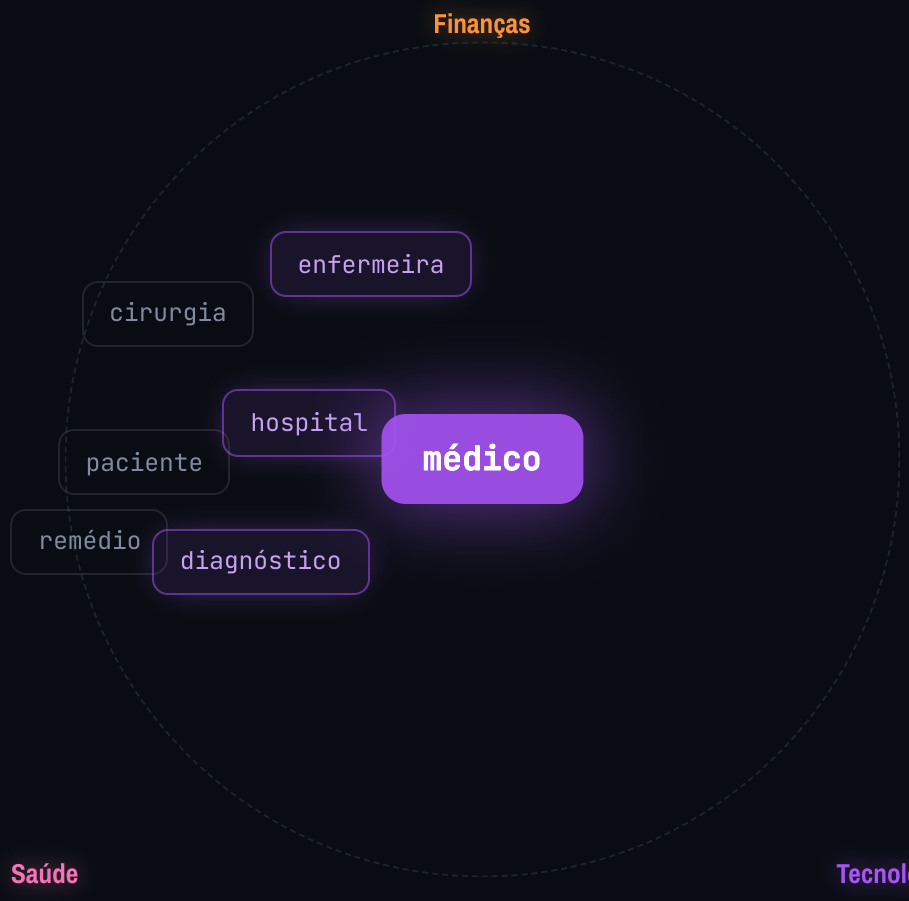
Retorna sequência mais provável

Resultado estatisticamente médio

Por isso o resultado sem contexto é genérico. Não é culpa do modelo: é exatamente o que ele foi projetado para fazer.

Cada token vira um ponto num mapa de significados

Digite uma frase... (ex: O médico chegou no hospital)



Saúde médico, hospital, enfermeira, diagnóstico...

Veículos carro, motor, freio, combustível...

Finanças banco, investimento, juros, crédito...

Tecnologia código, software, algoritmo, dados...

A mesma palavra, vetores diferentes dependendo do contexto

"Banco" não tem um único vetor fixo. O vetor é calculado dinamicamente com base nas palavras ao redor.

"fui ao banco sacar dinheiro"	"banco" → próximo de:	dinheiro	agência	conta
"sentei no banco da praça"	"banco" → próximo de:	praça	sentar	descanso
"banco de dados da aplicação"	"banco" → próximo de:	SQL	tabela	query

O contexto que você fornece não é apenas "mais informação": ele literalmente muda os vetores que o modelo calcula, alterando quais padrões são ativados e qual resultado é gerado.

Modelos maiores não alucinam menos: alucinam melhor

Um modelo pequeno que alucina produz algo obviamente errado. Um modelo grande que alucina produz algo fluente, coerente, detalhado e completamente plausível, que nunca existiu.

Modelo pequeno alucinando

```
"A função authenticate() recebe um parâmetro chamado xyz_token_value_hash_abc"
```

claramente inventado, fácil de perceber

Modelo grande alucinando

```
"A função authenticate() implementa o padrão RFC 7519 com validação HMAC-SHA256, seguindo a convenção estabelecida pelo seu time de usar prefixo Bearer no header Authorization"
```

soa completamente real, difícil de detectar

Confident hallucination: o modelo não sabe que não sabe. E quanto maior o modelo, mais convincente é a ficção que ele produz.

Modelos melhores não resolvem o problema do contexto

Toda semana sai um modelo novo. Mais parâmetros, melhor benchmark, mais capacidade. A tentação é esperar o modelo perfeito que vai acertar tudo. Esse modelo não existe.

✓ Com contexto

resultados mais precisos

`modelo + contexto = referência específica para verificar`

✗ Sem contexto

alucinações mais convincentes

`modelo sem referência = ficção plausível`

A evolução dos modelos melhora o teto do que é possível. O contexto determina se você chega perto desse teto ou fica na média.

RAG: buscar o que é relevante antes de responder

Uma base vetorial armazena documentos como vetores. Quando você faz uma pergunta, o sistema calcula a distância entre o vetor da sua pergunta e os vetores dos documentos armazenados e injeta os mais próximos no contexto do modelo.

Fluxo RAG

- Sua pergunta → transforma em vetor
- Busca os K vetores mais próximos na base de conhecimento
- Injeta os documentos correspondentes no contexto
- Modelo responde com o conhecimento específico recuperado

Sem RAG

Resposta com treinamento genérico.

Com RAG

Documentos do projeto injetados no contexto.

O modelo não fica mais inteligente. Ele fica mais informado: com o conhecimento específico do seu projeto injetado no momento certo.

Fine-tuning: ensinar o modelo a trabalhar do seu jeito

Um modelo base foi treinado com texto genérico — sabe um pouco de tudo. Fine-tuning é um treinamento adicional com exemplos do seu domínio. Você não treina do zero: refina o que ele já sabe para o seu contexto.

Modelo base

sabe um pouco de tudo

Fine-tuning com exemplos específicos

código Python, convenções do time, padrões da empresa

Modelo especializado

sabe muito sobre o seu domínio

É a diferença entre contratar um profissional generalista e um especialista que já trabalhou anos no seu setor.

Prompt é instrução. Contexto é memória.

Imagine contratar um desenvolvedor sênior para o seu projeto. No primeiro dia você explica tudo: a stack, as convenções, o que não pode fazer, como o sistema funciona.

Dia 1	Você explica tudo
Dia 2	Ele acorda sem nenhuma memória do que foi dito. Você explica tudo de novo.
Dia 3	O mesmo.

É exatamente assim que um agent funciona sem engenharia de contexto. Cada sessão começa do zero. O resultado depende inteiramente de quanto contexto você consegue repassar a cada vez.

Garantia de cumprimento da SPEC com Claude Code

Não existe percentual fixo. Depende de variáveis críticas.

✓ AUMENTA aderência	
SPEC clara, sem ambiguidade	Alto
Exemplos concretos e casos de borda	Alto
Tarefas menores e bem delimitadas	Alto
Revisão humana por checkpoint	Alto
TDD (testes guiam a implementação)	Alto
CLAUDE.md com regras do projeto	Médio

◎ DIMINUE aderência	
SPEC longa com requisitos implícitos	Alto
Requisitos contraditórios ou vagos	Alto
Tarefa muito grande em um único prompt	Alto
Sem revisão intermediária	Alto
Dependências externas não documentadas	Médio

~85-95%
SPEC simples + clara + pequena

~60-75%
SPEC média com detalhes implícitos

~30-50%
SPEC grande sem checkpoints

0%?
SPEC ambígua ou contraditória

Claude Code não é um contratado que entrega 100% no prazo. **É um colaborador que precisa de feedback contínuo para chegar lá.**

Sem SPEC (modo caótico)

- Agente começa sem saber o escopo
- Interpreta o objetivo e assume
- Codifica, refaz, diverge do problema
- 5-8 iterações para corrigir o caminho
- Entrega funcionalidades que ninguém pediu
- Resultado: retrabalho de 30-60% do código

VS

Com SPEC: SDD

- Agente lê a SPEC (2-3K tokens)
- Escopo claro, módulos definidos
- Implementa focado nos critérios de aceitação
- 1-2 iterações para refinar detalhes
- Entrega exatamente o que foi especificado
- Resultado: primeira versão já é utilizável

SPEC: o contrato entre sua ideia e o código

SPEC é o documento que traduz sua ideia em instruções claras para o agente IA

- Sem SPEC, o agente não tem escopo: começa codando, refaz, diverge, entrega errado
- Com SPEC, o agente sabe exatamente o que construir, em quais módulos e com qual stack
- ✓ SPEC é o seu TDD de alto nível: define critérios de aceitação antes da primeira linha de código
- Depois do próximo slide, você vai gerar a SPEC a partir da sua ideia validada

SPEC.md

```
# SPEC.md

## Problema
```
Descreva o problema que o projeto resolve em 2-3 frases.
Quem sofre com isso hoje? Por que é importante?
```

## Usuários
```
Quem vai usar? Qual o perfil?
```

## Funcionalidades
### Essenciais
- Funcionalidade 1
- Funcionalidade 2
- Funcionalidade 3

### Fora do escopo
```

O que vai dentro do arquivo

CLAUDE.md

```
# CLAUDE.md
## Stack

Backend: FastAPI · Frontend: Next.js · DB: SQLite · LLM: Groq

## Padrões
- Rotas REST em snake_case
- Componentes React em PascalCase
- Todo endpoint tem tipagem Pydantic
## TDD
- pytest para backend, Vitest para frontend
- Testes ficam em /tests espelhando a estrutura do módulo
- Nenhuma função de negócio sem teste
```

Context Exclusion File

O que o agente nunca deve ler

Todo agent de código lê o seu repositório. O problema é que ele não distingue **código de negócio** de **lixo, segredo ou ruído**. O Context Exclusion File lista o que deve ser invisível: dependências, builds, variáveis de ambiente, arquivos gerados. Sem ele, o agent desperdiça context window com `node_modules` e pode vazar segredos da sua `.env` para o modelo.

É o **`.gitignore`**, mas para a IA: o que o Git não versiona, o ignore file garante que o agente não lê.

Ferramenta	Arquivo	Status
Claude Code	`.claudeignore`	✓ disponível
OpenCode	`.gitignore`	⚠ sem arquivo próprio
Gemini CLI	`.geminiignore`	✓ disponível

Para OpenCode, a proteção hoje vem do .gitignore: certifique-se de que ele está bem configurado.

Claude Code e OpenCode: como configurar

Claude Code `.claudeignore`

```
# Dependências
node_modules/
.venv/
__pycache__/

# Build e gerados
dist/
build/
.next/
*.pyc

# Segredos e ambiente
.env
.env.*
*.pem
```

Claude Code: bug conhecido pode ignorar .claudeignore e ler .env. Nunca deixe segredos reais no repositório.

OpenCode `.gitignore`

```
# .gitignore — OpenCode respeita este arquivo
.env
.env.*
node_modules/
__pycache__/
*.sqlite
*.key
*.pem
dist/
build/
```

Sem arquivo próprio ainda. Proteção vem do .gitignore.

OpenCode: issue aberta pedindo .opencodeignore. Por enquanto .gitignore é a única proteção.

PLAN.md

O contrato de execução entre você e os agents

O PLAN.md é o documento que transforma intenção em execução estruturada. Sem ele, o agent lê o SPEC e **começa a implementar em qualquer ordem**, tomando decisões de arquitetura sozinho, criando conflitos com outros agents e quebrando dependências invisíveis. Com ele, cada agent sabe exatamente **o que fazer, em que ordem, quais arquivos tocar** e o que é esperado como output.

✎ Planejamento antes de execução

Agents são treinados para ser úteis, o que significa que começam a escrever código imediatamente. O PLAN.md é a restrição que impede isso.

▤ Contrato legível por máquina

Cada task tem input, output e testes definidos antes de qualquer código existir. O agent não toma decisões de arquitetura, ele segue o plano.

⌈ Gate entre fases

A próxima fase só começa quando os testes da anterior passam. Sem gate, erros se propagam silenciosamente entre tasks.

Plan Mode Claude Code tem modo nativo somente leitura onde o model analisa o repositório e gera o plano sem modificar nenhum arquivo. Use-o para gerar e revisar o PLAN.md antes de qualquer execução.

Sprint → Fase → Task: a estrutura que os agents seguem

```
Sprint
└ unidade de tempo com critério de entrega claro
   ex: "sistema funcionando end-to-end"
├ Fase 1 (sem dependências)
|  ├ Task 1.1 [Agent A]
|  ├ Task 1.2 [Agent B] ← paralelo
|  └ Task 1.3 [Agent C] ← paralelo
|
├ Fase 2 (depende da Fase 1 completa)
|  ├ Task 2.1 [Agent D]
|  └ Task 2.2 [Agent E] ← paralelo
|
├ Fase 3 (depende da Fase 2 completa)
|  └ Task 3.1 [Agent F]
```

Sprint

janela de tempo com um critério de entrega verificável. O que precisa estar funcionando ao final?

Fase

agrupamento de tasks que compartilham o mesmo pré-requisito. Tem um gate de teste antes de liberar a próxima.

Task

menor unidade de trabalho. Um agent, um escopo, um output. Deve ser concluída em uma única sessão.

O que pode rodar em paralelo e o que não pode

✓ Pode rodar em paralelo

- ✓ Tasks que tocam módulos ou pastas diferentes
- ✓ Tasks sem dependência de output uma da outra
- ✓ Tasks com testes próprios e independentes
- ✓ Tasks da mesma fase

✕ Não pode rodar em paralelo

- ✕ Task B que consome o output da Task A
- ✕ Duas tasks que editam o mesmo arquivo
- ✕ Tasks de integração antes das tasks base estarem prontas
- ✕ Tasks de fases diferentes

Regra simples

Se você consegue explicar a Task A sem mencionar a Task B, elas podem rodar em paralelo. Se não consegue, são sequenciais.

⚠ Erro mais comum

Colocar tasks de integração na mesma fase que as tasks que elas integram. O agent vai tentar **conectar coisas que ainda não existem**.

O teste vem antes: sempre

Em desenvolvimento com agents, TDD não é uma boa prática opcional: é o **mecanismo de controle**.
Sem um teste definido antes da execução, o agent decide sozinho o que "funcionando" significa.
Com o teste no PLAN.md, quem define o critério de sucesso é você, não o agent.



"Se o critério de sucesso não está escrito antes, o agent vai inventar um."

Red/Green: como fica no arquivo

Antes da execução `PLAN.md`

```
#### Task 2.1 — [nome]
- Testes críticos:
- [ ] comportamento com input válido
- [ ] caso de erro tratado
- [ ] caso limite coberto
```

`red` Agente escreve o teste

```
# red — teste existe, código não
# existe ainda

def test_comportamento_valido():
    resultado = funcao(input_valido)
    assert resultado == valor_esperado  # FALHA

def test_caso_de_erro():
    with pytest.raises(ErroEspecifico):
        funcao(input_invalido)  # FALHA
```

`green` Após implementação

```
#### Task 2.1 — [nome]
- Testes críticos:
- [x] comportamento com input válido
- [x] caso de erro tratado
- [x] caso limite coberto
```

Gate da fase: `pytest tests/modulo/ --tb=short` retorna 0 antes da próxima fase começar.

Toda task tem no mínimo dois testes: sem exceção

Teste 1: Comportamento esperado (happy path)

OBRIGATÓRIO

Dado: input válido e bem formado
Quando: a função/endpoint é chamado
Então: retorna o resultado correto no formato esperado

O que verifica: O agent implementou o que foi pedido, não uma versão aproximada.

Teste 2: Caso de erro tratado

OBRIGATÓRIO

Dado: input inválido, ausente ou malformado
Quando: a função/endpoint é chamado
Então: lança erro específico e conhecido. Nunca silencia nem retorna dado errado.

O que verifica: O agent não ignorou os edge cases para entregar mais rápido.

Tasks de integração: têm um terceiro teste obrigatório, o **teste de contrato**. Garante que o output da Task A é exatamente o que a Task B espera como input. É o que evita que dois agents construam interfaces incompatíveis em paralelo.

Uma task sem output definido não é uma task: é uma esperança

✕ Task mal escrita

```
#### Task 2.1 — Implementar o módulo de processamento
- Agent: Backend
- Fazer o processamento funcionar
- Testes: testar se funciona
```

Problemas:

- Output vago, agent toma todas as decisões de arquitetura
- Teste não é verificável
- Outro agent não sabe o que vai encontrar ao depender desta task

✓ Task bem escrita

```
#### Task 2.1 — [nome descritivo]
- Agent: [papel do agent]
- Input: [o que precisa estar pronto para começar]
- Output: [arquivo ou função específica com assinatura]
- Testes críticos:
- [ ] comportamento esperado com input válido
- [ ] caso de erro tratado corretamente
- [ ] caso limite coberto
```

Três perguntas para validar qualquer task antes de executar

1. Outro agent consegue usar o output desta task sem perguntar nada?
2. O critério de conclusão é verificável por um comando de terminal?
3. Os testes estão escritos antes do código?

Se alguma resposta for **não**, reescreva a task.

Scaffold: a fundação antes do código

O scaffold é a estrutura inicial do projeto: pastas, arquivos de configuração, dependências instaladas e servidores rodando. É criado antes de qualquer código de negócio. Sem ele, cada sub-agent cria sua própria estrutura ao começar uma task, gerando inconsistências impossíveis de reconciliar depois.

> *"Um diretório vazio equivale a um prompt vago. O agent não sabe se está construindo uma mansão ou uma cabana."*

✓ Consistência

Todos os agents trabalham na mesma estrutura de pastas, com as mesmas convenções de nomenclatura.

⌕ Contexto imediato

O agent lê o repositório e entende o projeto sem precisar de explicação. A estrutura scaffolded com organização clara ajuda agents futuros a tomar decisões mais informadas.

🔒 Gate de início

Nenhuma task de implementação começa sem o scaffold completo. Sem essa garantia, agents paralelos partem de premissas diferentes sobre onde os arquivos vão existir.

De pair programming para orquestração de equipe

Trabalhar com um único agent é **pair programming**: você guia, ele executa, o loop é síncrono e sequencial. O teto é a janela de contexto de uma sessão. Multi-agents é uma mudança de paradigma: você não escreve código, você orquestra uma equipe que escreve código.

*A diferença está entre o modelo **condutor**, onde você guia um único agent em tempo real, e o modelo **orquestrador**, onde você coordena um ensemble inteiro: múltiplos agents, cada um com sua janela de contexto, trabalhando de forma assíncrona.*

Antes	→	Agora
Escrever código	→	**Decompor trabalho com clareza**
Revisar linha a linha	→	**Verificar outputs por contrato**

As habilidades estão mudando, não sendo substituídas. Escrever código costumava ser a habilidade primária. Agora você precisa dominar delegação eficaz, revisão de código gerado por IA e coordenação de workflows paralelos.

Sequencial → Paralelo → Orquestrado: comece pelo mais simples

1 Sequencial por fase

Um agent, uma task por vez. Você valida o output de cada task antes da próxima começar. Mais controle, menos velocidade.

```
Você → Agent → Task 1.1 → valida
     → Task 1.2 → valida → Task 1.3
```

Quando usar: primeira fase da imersão, tasks com dependências fortes, quando quer mostrar o fluxo didaticamente.

2 Paralelo dentro da fase

Múltiplos agents simultâneos, cada um com uma task independente da mesma fase. O PLAN.md já mapeou o que pode rodar em paralelo.

```
      ┌→ Agent A → Task 2.1 ┐
Você ─┤  Agent B → Task 2.2 ├→ gate → Fase 3
      └→ Agent C → Task 2.3 ┘
```

Quando usar: tasks genuinamente independentes, módulos diferentes, sem edição dos mesmos arquivos.

3 Orquestrador + sub-agents

Um agent principal lê o PLAN.md e delega tasks para sub-agents automaticamente. Você fala com o orquestrador, ele coordena o resto.

```
Orquestrador lê o PLAN.md e delega
para sub-agents. Especialistas pegam
tasks do seu domínio, marcam progresso
e sinalizam outputs ao concluir.
```

Quando usar: quando o PLAN.md está sólido e você quer máxima autonomia.

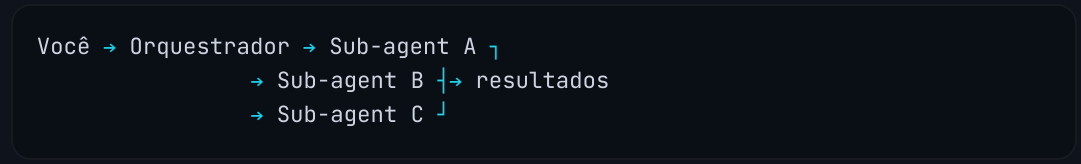
Sub-agents para delegar. Agent Teams para colaborar.

Os workers precisam se comunicar entre si?

Sub-agents

Workers focados que reportam ao principal

Roda dentro da sessão principal ou em sessões isoladas. Só reportam resultados ao agent principal: não se comunicam entre si diretamente.



Ideal para:

✓ Tasks paralelas independentes

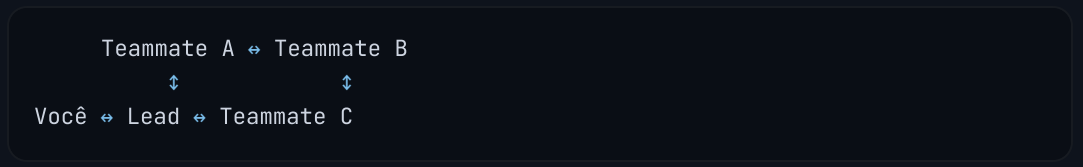
✓ Análise de módulos separados

✓ Implementação de features isoladas

Agent Teams

Múltiplas instâncias colaborando entre si

Lista de tasks compartilhada, rastreamento de dependências e comunicação peer-to-peer. Teammates compartilham descobertas e coordenam por conta própria.



Ideal para:

✓ Coordenação cross-layer (frontend + backend + testes)

✓ Debugging com hipóteses concorrentes

✓ Trabalho que abrange múltiplos domínios

No OpenCode: suporte a múltiplos providers no mesmo time. Claude para orquestração, GPT para raciocínio profundo, Gemini para frontend: tudo coordenado na mesma equipe.

Skills: o que são e por que existem

Você já usou o CLAUDE.md/AGENTS.md para dar contexto permanente ao agent. Skills resolvem um problema diferente: como fornecer ao Claude orientação especializada de domínio **exatamente quando necessário**, sem overhead permanente para tarefas não relacionadas?

	Quando carrega	Para que serve	Escopo	Quem mantém
`CLAUDE.md / AGENTS.md`	Sempre, em toda sessão	Convenções do projeto	Projeto inteiro	Você
`Prompt / Comando`	Quando você escreve	Tarefa pontual	Uma interação	Você
`Skill`	Sob demanda, quando relevante	Workflow especializado reutilizável	Uma classe de tarefa	Você ou comunidade

Uma skill pode conter referências, convenções ou instruções passo a passo. Exemplos:

Toda vez que um endpoint FastAPI criado

seguir convenções sem repetir no prompt

Toda vez que um teste escrito

seguir padrão TDD definido no CLAUDE.md

Toda vez que um componente React gerado

seguir design system do projeto

Skills são conteúdo de referência ou instruções passo a passo que o Claude aplica sob demanda, sem poluir o contexto de tarefas não relacionadas.

SKILL.md: a estrutura por dentro

Toda skill tem exatamente esta estrutura:

Estrutura de pastas

```
minha-skill/ ← pasta com o nome da skill
 └ SKILL.md ← arquivo obrigatório
 └ scripts/ ← opcional: scripts executáveis
 └ references/ ← opcional: docs sob demanda
 └ assets/ ← opcional: templates, arquivos
```

SKILL.md

duas partes: frontmatter + instruções

frontmatter (metadata)

```
---
name: nome-da-skill
description: O que a skill faz e
quando usar. Esta é a parte mais
importante — o Claude lê isso para
decidir se deve carregar a skill.
---
```

corpo (instruções)

```
# Título da Skill

Instruções em markdown que o
Claude segue quando a skill é
invocada.

## Passo 1
Faça X...

## Passo 2
Faça Y...
```

Regras do frontmatter:

name	máximo 64 caracteres, apenas letras minúsculas, números e hífens
description	máximo 1024 caracteres, escreva em terceira pessoa, inclua o que faz e quando usar

O que realmente importa:

O frontmatter não é instrução. É metadata. O campo description é o que o Claude lê para decidir se a skill é relevante para a requisição.

Como o agent encontra e usa a skill

Startup — Claude lê frontmatter de todas as skills (~100 tokens/skill)

↓

Você faz uma requisição

↓

Claude verifica relevância — alguma skill é relevante?

↓

Carrega sob demanda — Sim → SKILL.md completo | Não → segue sem carregar

↓

Só o necessário — `references/` e `scripts/` apenas se mencionados nas instruções

Automático

O Claude detecta que a tarefa é relevante e carrega a skill sem você pedir. Funciona bem quando a description é específica e bem escrita.

Manual

```
# Claude Code  /nome-da-skill
# OpenCode     use skill nome-da-skill
```

⚠ **O Claude tende a "under-trigger" skills: não usá-las quando seriam úteis.** Escreva a description um pouco "insistente": em vez de "Como fazer X", escreva "Use esta skill sempre que o usuário mencionar X, Y ou Z, mesmo que não peça explicitamente."

Onde criar: projeto vs global

Claude Code

Projeto *(só neste projeto)*

```
.claude/skills/nome-da-skill/SKILL.md
```

Global *(todos os projetos)*

```
~/.claude/skills/nome-da-skill/SKILL.md
```

OpenCode

ordem de precedência

1. **Projeto** `.opencode/skills/nome-da-skill/SKILL.md`
2. **Global OpenCode** `~/.config/opencode/skills/nome-da-skill/SKILL.md`
3. Compat Claude `.claude/skills/nome-da-skill/SKILL.md`
4. Compat Claude `~/.claude/skills/nome-da-skill/SKILL.md`

Verificar

```
# Claude Code
/skills

# OpenCode
"Quais skills você tem disponíveis?"
```

Regra simples: skills específicas do projeto ficam na raiz do projeto. Skills que você usa em todos os projetos ficam em `~/.claude/skills/` ou `~/.config/opencode/skills/`.

Agents: personas especializadas com escopo definido

Você criou skills que ensinam o agent a fazer coisas específicas. Agora vamos criar agents que são personas com identidade, ferramentas restritas e skills associadas: cada um responsável por um domínio do projeto.

Agent primário

`Shift+Tab` `Tab`

com quem você conversa diretamente. Orquestra o trabalho, decide quem faz o quê.

Sub-agent

`@nome` `@nome`

worker especializado invocado para uma task. Roda com contexto isolado, ferramentas restritas e retorna o resultado.

Por que isso importa para o projeto:

enforçam least privilege

agents com acesso restrito a ferramentas são mais confiáveis que uma sessão onipotente

contexto isolado

o sub-agent trabalha independentemente sem contaminar a conversa primária

persona com identidade

cada agent tem system prompt próprio, skills associadas e domínio definido

Sub-agents são agents independentes com system prompt próprio, acesso a tools e janela de contexto isolada. Quando o agent principal delega uma task, o sub-agent trabalha independentemente e retorna o resultado.

Agent file: a estrutura por dentro

Um agent é um arquivo Markdown com **frontmatter YAML** + **system prompt** no corpo:

Claude Code

`.claude/agents/parser-agent.md`

frontmatter

```
---
name: parser-agent
description: Extrai e processa texto de
arquivos PDF e DOCX. Use quando precisar
fazer parsing de currículos ou documentos
enviados pelo usuário.
tools: Read, Write, Bash
model: sonnet
---
```

corpo (system prompt)

```
Você é um especialista em extração de texto
de documentos.

## Responsabilidades
- Extrair texto de PDFs e DOCX
- Normalizar o texto extraído
- Salvar no formato do CLAUDE.md

## Restrições
- Nunca acesse o banco diretamente
- Nunca modifique fora de /services/
- Nunca execute cmds não relacionados
```

OpenCode

`.opencode/agents/parser-agent.md`

frontmatter

```
---
description: Extrai e processa texto de
arquivos PDF e DOCX.
mode: subagent
model: anthropic/claude-sonnet-4-20250514
temperature: 0.1
tools:
read: true
write: true
bash: true
edit: false
---
```

corpo

```
Você é um especialista em extração de texto
de documentos...
```

Diferença chave: OpenCode usa `mode: subagent` e tools granulares (read/write/bash/edit true/false). Claude Code lista tools como string e aceita `model: sonnet` como alias.

Cada agent recebe só o que precisa

Cada sub-agent tem permissões mínimas necessárias: agents somente leitura como revisores recebem apenas `Read, Grep, Glob`. Agents de código recebem `Read, Write, Edit, Bash`. Agents de pesquisa recebem `WebFetch, WebSearch`.

Tipo de agent	Tools permitidas	Tools negadas
Revisor de código	`Read` `Grep` `Glob`	~~`Write`~~ ~~`Edit`~~ ~~`Bash`~~
Parser de arquivos	`Read` `Write` `Bash`	~~`Edit`~~ ~~`WebFetch`~~
Analista LLM	`Read` `Write` `Bash`	~~`Delete`~~ ~~`Git`~~
Frontend	`Read` `Write` `Edit`	~~`Bash direto em DB`~~
Testes/QA	`Read` `Bash (test only)`	~~`Write em prod`~~

Além de segurança: um agent com escopo restrito toma decisões mais focadas porque não precisa considerar o projeto inteiro, só o seu domínio.

Onde criar: projeto vs global

Claude Code

Projeto *(versionado com o código)*

```
.claude/agents/nome-do-agent.md
```

Global *(todos os projetos)*

```
~/.claude/agents/nome-do-agent.md
```

Agents em `.claude/agents/` são versionados no controle de versão para que o time use e melhore colaborativamente.

OpenCode

Projeto `.opencode/agents/nome-do-agent.md`

Global `~/.config/opencode/agents/nome-do-agent.md`

Criar (Claude Code)

```
# Claude Code
/agents
# → Create new agent
# → Project ou Personal
# → Generate with Claude ou Manual
```

Listar agents

```
# Claude Code
claude agents

# OpenCode
"Quais agents você tem disponíveis?"
```

Regra: agents específicos do projeto em `.claude/agents/` ou `.opencode/agents/`. Agents reutilizáveis em `~/.claude/agents/` ou `~/.config/opencode/agents/`.

Agent + Skill: especialização em dois níveis

Skills ensinam o agent o que fazer. Agents definem quem faz e com quais permissões. Juntos criam o sistema mais eficaz.

Claude Code

injetar skill no agent

```
---
name: backend-agent
description: Implementa endpoints FastAPI e lógica de serviço.
Use para criar rotas, modelos e services do backend.
tools: Read, Write, Edit, Bash, Glob, Grep
model: sonnet
skills:
- api-conventions
- tdd-backend
---

Você é um engineer especializado em backend Python.
Siga sempre as convenções do CLAUDE.md.
```

OpenCode

skill no agent

```
---
description: Implementa endpoints FastAPI e lógica de serviço.
mode: subagent
model: anthropic/claude-sonnet-4-20250514
temperature: 0.2
tools:
read: true
write: true
edit: true
bash: true
---

Você é um engineer especializado em backend Python.
Leia e aplique a skill de api-conventions do projeto.
Siga as convenções do AGENTS.md.
```

Claude Code: o campo `skills:` injeta o conteúdo da skill no contexto do sub-agent no startup. Isso dá ao sub-agent conhecimento de domínio sem precisar descobrir e carregar skills durante a execução.

Invocando agents: manual e automático

Invocação manual

```
# Claude Code e OpenCode — @ mention

› @parser-agent Processe os currículos em /uploads/

› @backend-agent Implemente a Task 2.2 do PLAN.md

› @qa-agent Rode os testes do módulo de autenticação
e reporte o que está falhando
```

Delegação automática pelo agent primário

```
Leia o PLAN.md e execute a Fase 3.
Para cada task, delegue ao agent especializado:
- Tasks de parsing → @parser-agent
- Tasks de API → @backend-agent
- Tasks de frontend → @frontend-agent
- Validação de testes → @qa-agent

Cada agent roda com contexto isolado.
Aguarde o retorno de cada um antes de avançar
para tasks com dependência.
```

O agent primário orquestra, os sub-agents executam. O contexto de cada sub-agent é limpo: sub-agents não herdam o system prompt completo. Eles recebem seu próprio prompt mais detalhes básicos de ambiente, o que os torna mais fáceis de moldar deliberadamente.

Hooks no frontmatter: guardrails que vivem dentro do agent

No bloco de Review e Entrega vimos hooks globais, configurados no settings.json e ativos para toda sessão. Agora temos agents com escopos definidos. O próximo nível é colocar os hooks dentro do próprio agent: eles só disparam quando aquele agent específico está rodando.

Hook global (settings.json)	Hook no agent (frontmatter)
Dispara em toda sessão	**Dispara só neste agent**
Regras gerais do projeto	**Regras específicas do domínio**
Formato e lint universal	**Validação por responsabilidade**
Um único ponto de configuração	**Cada agent cuida de si mesmo**

Importante: hooks no frontmatter do agent disparam quando o agent é invocado como sub-agent via Task tool ou @mention. Não disparam quando o agent roda como sessão principal. Para hooks de sessão completa, configure em settings.json.

SubagentStop: stop hooks definidos no frontmatter de agents são automaticamente convertidos em eventos SubagentStop. Quando o sub-agent encerrar, o hook roda antes de retornar o resultado ao agent principal.

Como declarar hooks dentro do agent: Claude Code

A estrutura é simples: a seção hooks fica no frontmatter do arquivo `.claude/agents/nome-do-agent.md`

.claude/agents/backend-agent.md

```
---
name: backend-agent
description: Implementa endpoints FastAPI e lógica de serviço.
tools: Read, Write, Edit, Bash, Glob, Grep
model: sonnet
hooks:
PreToolUse:
- matcher: "Bash"
  hooks:
  - type: command
    command: ".claude/hooks/backend/validate-cmd.sh"
PostToolUse:
- matcher: "Edit|Write"
  hooks:
  - type: command
    command: ".claude/hooks/backend/run-tests.sh"
Stop:
- hooks:
  - type: command
    command: ".claude/hooks/backend/verify-conventions.sh"
```

PreToolUse

Roda antes da tool executar. Use matcher para filtrar quais tools disparam o hook.

`matcher: "Bash"`

PostToolUse

Roda após a tool executar. Ideal para lint, testes, validação.

`matcher: "Edit|Write"`

Stop → SubagentStop

Roda antes do sub-agent encerrar e retornar resultado ao agent principal.

`converte automaticamente`

OpenCode: hooks ficam em plugins TypeScript em `.opencode/plugin/`, não no frontmatter do agent. A lógica é a mesma, a implementação é diferente.

Cada agent tem os guardrails do seu domínio

O valor real dos hooks por agent é que cada um enforça as regras específicas da sua responsabilidade.

parser-agent

Valida formato do output

PostToolUse · Write

```
.claude/hooks/parser/validate-output.sh
```

Stop

```
.claude/hooks/parser/check-encoding.sh
```

backend-agent

Roda testes após cada edição

PostToolUse · Edit|Write

```
pytest tests/ -q --tb=line 2>&1 | tail -10; exit 0
```

PreToolUse · Bash

```
.claude/hooks/backend/block-db-writes.sh
```

code-reviewer

Somente leitura, bloqueia qualquer escrita

PreToolUse · Write|Edit|Bash

```
echo 'Agente de review não pode modificar arquivos' >&2 && exit 2
```

qa-agent

Só permite comandos de teste

PreToolUse · Bash

```
.claude/hooks/qa/allow-only-tests.sh
```

Os scripts que cada hook chama

Os hooks chamam scripts que ficam em `.claude/hooks/nome-do-agent/`

Bloquear comandos fora do escopo

backend-agent · .claude/hooks/backend/block-db-writes.sh

```
#!/bin/bash
# .claude/hooks/backend/block-db-writes.sh
INPUT=$(cat)
CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# Bloquear DROP, DELETE direto, push para main
if echo "$CMD" | grep -qE '(DROP TABLE|DELETE FROM|git push origin main)'; then
echo "Comando bloqueado: fora do escopo" >&2
exit 2
fi
exit 0
```

Validar output do parser

parser-agent · .claude/hooks/parser/validate-output.sh

```
#!/bin/bash
# .claude/hooks/parser/validate-output.sh
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

# Verificar se arquivo de output tem conteúdo
if [ -f "$FILE" ] && [ ! -s "$FILE" ]; then
echo "ERRO: parser gerou arquivo vazio" >&2
exit 2
fi
exit 0
```

Verificar convenções (Stop → SubagentStop)

backend-agent · .claude/hooks/backend/verify-conventions.sh

```
#!/bin/bash
# .claude/hooks/backend/verify-conventions.sh
INPUT=$(cat)
# Evitar loop infinito
[ "$(echo "$INPUT" | jq -r '.stop_hook_active')" = "true" ] && exit 0

# Rodar pytest nos arquivos modificados
pytest tests/ -q --tb=short 2>&1 | tail -5
exit 0 # não bloquear — só reportar
```

Rules: instruções modulares que o agent lê quando relevante

Você tem o CLAUDE.md/AGENTS.md com as convenções gerais. Mas à medida que o projeto cresce, esse arquivo cresce junto. Chegou a hora de modularizar.

Três camadas de contexto

CLAUDE.md / AGENTS.md

Carrega: Sempre, toda sessão
Para que: Visão geral do projeto
Tamanho: Até 200 linhas
Escopo: Todo o projeto

Rules

Carrega: Sempre ou por path
Para que: Restrições específicas
Tamanho: Uma regra por arquivo
Escopo: Por domínio ou path

Skills

Carrega: Sob demanda
Para que: Workflows completos
Tamanho: Sem limite prático
Escopo: Por classe de tarefa

Rule vs Hook

	Hook	Rule
O que é	Script externo executado	Instrução que o agent lê
Quando dispara	Em eventos específicos	Quando o arquivo é carregado
Pode bloquear	Sim, com exit 2	Não, só orienta
Pode ser ignorado	Não	Sim, se o agent julgar irrelevante
Melhor para	Segurança determinística	Comportamento contextual

Rules incondicionais vs escopadas por path

Dois tipos de rules no Claude Code. A distinção importa para economia de contexto.

Incondicional

Sem frontmatter de paths. Carrega em toda sessão.

```
<!— .claude/rules/seguranca.md —>
# Regras de Segurança

- Nunca commitar arquivos .env ou .env.*
- Nunca expor API keys em logs ou mensagens
- Nunca rodar migrations em produção sem
  aprovação explícita
- Sempre usar variáveis de ambiente para
  configuração sensível
```

Escopada por path

Só carrega quando o Claude trabalha nos paths correspondentes.

```
<!— .claude/rules/backend-api.md —>
---
paths:
- "backend/app/api/**/*.py"
- "backend/app/routes/**/*.py"
---
# Regras de API

- Toda rota deve ter tipagem Pydantic no request e response
- Retornar sempre { data, error }
- Incluir comentários OpenAPI em toda rota pública
- Nunca expor IDs de banco diretamente
- usar UUIDs públicos
```

Glob patterns: path-scoped rules disparam quando o Claude lê arquivos correspondentes ao padrão. Use `**/*.py`, `src/**` ou qualquer combinação.

Organizando rules por módulo do projeto

A forma mais eficaz de usar rules em um projeto com multi-agents é organizar por domínio: cada módulo com suas próprias regras.

.claude/rules/

```
.claude/
  └ rules/
      ├ seguranca.md  incondicional
      ├ convencoes-gerais.md  incondicional
      ├ backend/
      │   ├ api.md  paths: backend/app/api/**
      │   ├ models.md  paths: backend/app/models/**
      │   └ services.md  paths: backend/app/services/**
      ├ frontend/
      │   ├ components.md  paths: frontend/src/components/**
      │   └ hooks.md  paths: frontend/src/hooks/**
      └ tests/
          └ padroes.md  paths: tests/**
```

Incondicionais

Carregam em toda sessão junto com o CLAUDE.md. Use para restrições universais.

Backend

Só carregam quando o agent trabalha em arquivos Python do backend. Tipagem Pydantic, padrões de rota.

Frontend

Só carregam quando o agent trabalha em componentes React. Padrões de componente e hooks.

Testes

Só carregam quando o agent trabalha na pasta de testes. Padrões de teste e cobertura.

Contexto limpo: o agent backend recebe as rules de `backend/api.md` quando trabalha em rotas, mas não carrega as rules de frontend.

Como rules e agents se complementam

Skills ensinam o que fazer. Agents definem quem faz com qual escopo. Hooks enforçam via código. Rules orientam o comportamento dentro do domínio.

A tríade completa de controle

frontmatter do agent

- ↓ tools — o que pode usar
- ↓ model — com qual capacidade
- ↓ hooks — o que é determinístico

system prompt do agent

- ↓ quem ele é
- ↓ responsabilidades

rules do domínio (carregadas por path)

- ↓ como deve se comportar neste módulo
- ↓ restrições específicas do contexto

Exemplo prático

`backend-agent edita backend/app/api/routes.py`

Frontmatter	define que ele tem Read, Write, Edit, Bash
Hook PostToolUse	roda pytest após a edição
Rule backend/api.md	já está no contexto: toda rota precisa de tipagem Pydantic
System prompt	lembra que ele é especialista em backend Python

Nenhuma dessas camadas sozinha dá o controle total. **Juntas, definem o comportamento sem ambiguidade.**

Duas formas de rodar o reviewer automaticamente

Opção 1: o agent de implementação dispara o reviewer ao encerrar via Stop hook no frontmatter.

.claude/agents/backend-agent.md

```
---
name: backend-agent
description: Implementa endpoints e serviços.
tools: Read, Write, Edit, Bash, Glob, Grep
model: sonnet
hooks:
Stop:
- hooks:
- type: command
command: ".claude/hooks/backend/trigger-review.sh"
---
```

trigger-review.sh

```
#!/bin/bash
# .claude/hooks/backend/trigger-review.sh
INPUT=$(cat)
[ "$(echo "$INPUT" | jq -r '.stop_hook_active')" = "true" ] && exit 0

echo "Disparando review..." >&2
claude --agent code-reviewer \
"Faça o review do que foi implementado agora"
exit 0
```

Limitação: o reviewer roda dentro da mesma sessão, não é contexto completamente isolado. Para isolamento total, use a Opção 2.

OpenCode: plugin que dispara o reviewer após cada turno

.opencode/plugin/auto-review.ts

```ts
// .opencode/plugin/auto-review.ts
import type { Plugin } from "@opencode-ai/plugin"

export const AutoReview: Plugin = async ({ client }) => {
  return {
    "event": async ({ event }) => {
      if (event.type === "session.idle") {
        const agentName =
          (event as any).agentName

        // só dispara após agents de implementação
        const implAgents = [
          "backend", "frontend", "parser"
        ]

        if (implAgents.includes(agentName)) {
          await client.session.prompt({
            text: `@code-reviewer faça o review
                   do que foi implementado agora
                   usando o SPEC.md como ref.`,
            noReply: false
          })
```

Hooks globais: o que enforça a sessão inteira

Os hooks por agent enforçam regras do domínio de cada agent. Os hooks globais enforçam regras da sessão inteira: rodam independente de qual agent está ativo, independente do que foi feito.

	`Frontmatter do agent`	`settings.json`
Escopo	Só quando aquele agent roda	Toda sessão
Quem configura	Por agent	Uma vez, para o projeto
Uso típico	Regras de domínio	Gates de encerramento

Próximo passo: três hooks de encerramento no settings.json que garantem que nenhuma sessão termina sem testes passando e tasks concluídas.

OpenCode: equivalente em `.opencode/plugin/session-close.ts` usando o evento session.idle para disparar os mesmos gates.

Slash commands: encapsulando workflows em um comando

Você passou o dia inteiro construindo prompts longos e precisos. Um slash command transforma esse prompt em um único comando que qualquer um do time executa da mesma forma, toda vez.

Sem slash command	Com slash command
Prompt longo com contexto manual	`/review`
Lembrar qual agent usar	`Já está encapsulado`
Lembrar quais arquivos passar	`Já está encapsulado`
Sequência depende de você	`Sequência enforçada`

Claude Code: slash commands ficam em `.claude/skills/` como arquivos Markdown.

OpenCode: em `.opencode/commands/`.

RAG: dar memória específica ao modelo sem retreinar

O modelo foi treinado com conhecimento geral até uma data de corte. Ele não sabe nada sobre o seu projeto, as suas decisões, os bugs que você resolveu ontem.

RAG (Retrieval-Augmented Generation) resolve isso sem fine-tuning. Antes de responder, o sistema busca os documentos mais relevantes na sua base de conhecimento e os injeta no contexto do modelo.

Sem RAG

```
pergunta → modelo responde com treinamento geral

→ não sabe nada sobre o seu projeto
```

Com RAG

```
pergunta → busca documentos relevantes

  → injeta no contexto

  → modelo responde com conhecimento específico
```

O modelo não ficou mais inteligente. Ele ficou mais informado: com o conhecimento certo, no momento certo.

O fluxo completo: da pergunta à resposta fundamentada

1 INDEXAÇÃO
acontece antes, uma vez

Documento → chunks → embeddings → banco vetorial

2 RECUPERAÇÃO
acontece a cada pergunta

Pergunta → embedding da pergunta

→ busca vetores mais próximos no banco

→ recupera os K chunks mais relevantes

3 GERAÇÃO
o modelo responde

Prompt original + chunks recuperados

→ modelo gera resposta

→ fundamentada no conhecimento específico

A chave está na busca por **similaridade vetorial**. A pergunta e os documentos vivem no mesmo espaço vetorial: documentos semanticamente próximos da pergunta são os mais relevantes, independente de terem as mesmas palavras exatas.

Chunk size: o parâmetro que mais afeta a qualidade

Antes de gerar embeddings, os documentos são quebrados em **chunks**: pedaços menores. O tamanho do chunk define a granularidade da busca.

Chunk muito grande

1000+ tokens

- → embedding representa muitas ideias
- → busca retorna contexto amplo mas impreciso
- → model recebe muito ruído junto com o relevante

ruído

Chunk muito pequeno

50 tokens

- → embedding representa uma frase isolada
- → busca retorna fragmentos sem contexto suficiente
- → model recebe informação sem contexto para usar

fragmentado

Chunk ideal

200-400 tokens

- → embedding representa uma ideia completa
- → busca retorna contexto preciso e utilizável
- → model recebe exatamente o que precisa

preciso

Para o projeto da imersão: resumos de aprendizado em markdown. **Chunks de 200-300 tokens por seção funcionam bem.**

Construindo memória persistente para o projeto

O problema que o RAG resolve no projeto:

Sem memória persistente

Sessão 1:

- você resolve um bug complexo de autenticação
- descobre um limite de requisições na API
- decide uma abordagem de cache
- **sessão termina: model esquece tudo**

Sessão 2:

- model não sabe do limite
- comete o mesmo erro
- você explica de novo
- **tokens gastos, tempo perdido**

Com RAG do projeto

Sessão 1 termina:

- hook captura os aprendizados
- gera resumo em .claude/knowledge/
- faz ingest no banco vetorial

Sessão 2 começa:

- hook busca conhecimento relevante
- injeta no contexto antes do primeiro prompt
- **model já sabe do limite, da decisão do cache, do bug que foi resolvido**

Stack e estrutura do RAG do projeto

Stack:

SQLite + sqlite-vec

banco vetorial embedded, sem servidor, versionável

@xenova/transformers

embeddings locais, modelo all-MiniLM-L6-v2, ~25MB, sem API externa

Estrutura de pastas:

```
.claude/
├── knowledge/
│   ├── 2026-04-20-parser-rate-limit.md
│   ├── 2026-04-20-backend-decisao-cache.md
│   └── 2026-04-20-session-bug-autenticacao.md
├── rag.db
├── hooks/
│   └── global/
│       ├── capture-agent-learning.sh
│       ├── capture-session-learning.sh
│       └── inject-context.sh
└── scripts/
    ├── summarize.ts
    ├── embed.ts
    └── search.ts
```

MCP: o protocolo que conecta agents a ferramentas externas

Por padrão um agent só consegue ler e escrever arquivos, executar comandos e fazer buscas. O MCP expande isso: conecta o agent a qualquer ferramenta externa. Padrão aberto criado pela Anthropic, adotado por OpenAI, Google e Microsoft, doado à Linux Foundation.

Sem MCP	Com MCP
Agent lê arquivos	Agent consulta banco de dados
Agent executa bash	Agent controla um browser
Agent chama APIs via código	Agent usa ferramentas tipadas

Compatível: Claude Code, OpenCode, Cursor, Copilot e qualquer agent que implemente o protocolo.

Por dentro de um MCP: servidor, tools e protocolo

Um servidor MCP expõe tools que o agent pode chamar. O agent vê as tools disponíveis, decide qual usar e chama com os argumentos corretos: exatamente como chama Read ou Write, mas para ferramentas externas.

Fluxo de chamada



Tipos de servidor

Local

Roda na máquina do desenvolvedor (ex: Playwright)

Remoto

Roda em servidor externo (ex: integrações SaaS)

In-process

Roda dentro do próprio agent

Usando um MCP existente: Playwright ao vivo

Claude Code

```
claude mcp add playwright -- npx @playwright/mcp@latest

# verificar
/mcp
# → playwright ✓ connected
```

OpenCode — opencode.json

```
{
  "mcp": {
    "playwright": {
      "type": "local",
      "command": ["npx", "@playwright/mcp@latest"],
      "enabled": true
    }
  }
}
```

Teste ao vivo

```
navegue até localhost:3000 e tire um screenshot da página principal
```

O agent vai usar as tools do Playwright MCP para abrir o browser, navegar e retornar o screenshot: sem você escrever nenhum código.